

Итоговое задание: Многопользовательский табличный процессор на React и TypeScript

Обзор проекта

В рамках задания разрабатывается только фронтенд сервиса. Логике выдачи и проверки токенов, работы с базой данных реализовывать не нужно. Корректная работа механизма аутентификации и авторизации, сохранения/загрузки документов демонстрируется созданием тестов с заглушками, имитирующими сервер.

Цель

Разработать многопользовательское веб-приложение типа «табличный процессор» (spreadsheet). Каждый зарегистрированный пользователь работает только со своими документами — чужие документы недоступны. Приложение поддерживает создание, редактирование и удаление личных таблиц с богатой функциональностью ячеек.

1. Базовый табличный процессор (UI + логика ячеек)

Цель

Реализовать интерактивную таблицу с возможностью редактирования ячеек, базовыми формулами и управлением структурой (строки/столбцы).

Функциональные требования

- Рендеринг таблицы размером N×M (по умолчанию 26 столбцов × 100 строк), заголовки столбцов A–Z, строк — числа 1–100
- Выбор ячейки по клику, выделение диапазона с зажатым Shift
- Редактирование ячейки двойным кликом или нажатием **Enter** (инлайн-редактор)
- Поддержка типов данных: строка, число, логическое значение, формула (начинается с =)
- Базовые формулы: `=SUM(A1:A5)`, `=AVERAGE(B1:B3)`, `=A1+B1`, `=A1*2`
- Панель формул над таблицей: показывает содержимое активной ячейки
- Добавление/удаление строк и столбцов через контекстное меню (правая кнопка мыши)
- Изменение ширины столбца/высоты строки перетаскиванием

Критерии приёма

- Клик на ячейку — выделение; двойной клик — режим редактирования
- Ввод `=SUM(A1:A3)` вычисляет и отображает результат
- Добавление строки через контекстное меню вставляет новую строку без полной перерисовки таблицы
- Таблица из 1000 строк рендерится без видимых задержек (виртуализация)
- Ширина столбцов меняется перетаскиванием

2. Управление личными документами и сохранение

Цель

Реализовать полноценный CRUD для личных документов пользователя: создание, переименование, удаление, автосохранение и экспорт данных.

Функциональные требования

- Управление документами:
 - Дашборд со списком документов текущего пользователя: название, дата создания, дата последнего изменения, превью (первые 3×3 ячейки)
 - Создание нового документа: модальное окно с полями «Название» и «Начальный размер» (строки × столбцы)
 - Переименование документа инлайн или через кнопку
 - Удаление документа с подтверждением
 - Дублирование документа
- Сохранение и синхронизация:
 - Автосохранение: изменения ячеек дебаунсятся (500 мс) и отправляются на API `PATCH /documents/:id`
 - Индикатор статуса: «Сохранено», «Сохранение...», «Ошибка сохранения»
 - Ручное сохранение горячей клавишей `Ctrl+S`
 - При закрытии вкладки с несохранёнными изменениями — `beforeunload`
- Экспорт и импорт:
 - Экспорт таблицы в CSV
 - Экспорт в JSON
 - Импорт CSV с отображением названий колонок

Критерии приёма

- Пользователь видит список только своих документов
- Изменение ячейки автоматически сохраняется в течение 500 мс
- Экспорт в CSV скачивается и корректно открывается в Excel / Google Sheets
- Импорт CSV из 500 строк загружается без зависания
- Создание, переименование, дублирование, удаление документов работают корректно

3. Внедрение Redux Toolkit

Цель

Перенести всё глобальное состояние приложения из локальных `useState/useReducer/Context` в Redux Toolkit. Рефакторинг не должен ломать существующую функциональность.

Требования

- Архитектурные изменения:
 - Установка и настройка Redux Toolkit (`@reduxjs/toolkit`) и `react-redux`

- Типизация `RootState` и `AppDispatch`; экспорт хуков `useAppSelector`, `useAppDispatch`
 - Разделение стора на слайсы:
 - `spreadsheetSlice` — ячейки, выделение, история изменений (Undo/Redo)
 - `documentsSlice` — список документов, статус загрузки, активный документ
 - `uiSlice` — модальные окна, уведомления, статус сохранения
 - `authSlice` — подготовить структуру заранее, но временно хранить mock-пользователя до недели 5
 - Реализация Undo/Redo через `spreadsheetSlice`
 - Thunks (`createAsyncThunk`) для асинхронных операций: загрузка списка документов, загрузка документа, сохранение
 - Автосохранение реализуется через middleware с дебаунсом
- Требования к рефакторингу:
 - Все глобальные данные, кроме локального UI конкретной ячейки, хранятся в Redux
 - Покрыть слайсы unit-тестами
 - Не добавлять новую функциональность — только рефакторинг

Критерии приёма

- Глобальное состояние находится в Redux Store
- В Redux DevTools видны все основные действия
- Undo (`Ctrl+Z`) отменяет последнее изменение ячейки
- Redo (`Ctrl+Y`) повторяет отменённое изменение
- Существующие функции (документы, редактирование, сохранение) работают без регрессий
- Редюсеры всех слайсов покрыты unit-тестами

4. Внедрение React Router

Цель

Внедрить клиентскую маршрутизацию с помощью React Router, организовать структуру страниц и подготовить маршруты к последующему подключению авторизации.

Требования

- Архитектурные изменения:
 - Установка `react-router-dom`
 - Определение структуры маршрутов:

Маршрут	Компонент	Доступ
/	Редирект на <code>/dashboard</code>	Публичный
<code>/dashboard</code>	<code>DashboardPage</code>	mock-auth
<code>/documents/:documentId</code>	<code>SpreadsheetPage</code>	mock-auth
<code>/profile</code>	<code>ProfilePage</code>	mock-auth
*	<code>NotFoundPage</code>	Публичный

- Реализация layout-маршрутов: `AppLayout` (шапка + боковая панель)
- Использование `useNavigate`, `useParams`, `useLocation`
- Передача `documentId` из URL-параметра в `documentService`
- Подготовка обёртки `ProtectedRoute`
- Навигационные паттерны:
 - Хлебные крошки: `Мои документы` → `[Название документа]`
 - Подтверждение перехода при наличии несохранённых изменений (`useBlocker`)
 - Обработка 404 для несуществующего документа

Критерии приёма

- Переход по прямой ссылке `/documents/abc123` открывает документ
- `documentId` корректно извлекается из URL
- При попытке покинуть страницу с несохранёнными изменениями появляется диалог подтверждения
- Все существующие функции (стор, документы, редактирование) работают без регрессий
- Архитектура маршрутов готова к подключению аутентификации на следующем этапе

5. Аутентификация и авторизация

Цель

Подключить полноценную систему входа/регистрации и перевести приложение с mock-пользователя на реальную аутентификацию и авторизацию.

Функциональные требования

- Страница регистрации с полями: имя, email, пароль, подтверждение пароля
- Страница входа с полями: email, пароль
- Валидация форм на фронтенде (формат email, минимальная длина пароля ≥ 8 символов, совпадение паролей)
- Получение JWT Access Token и Refresh Token от API после успешного входа
- Хранение Access Token в памяти (in-memory), Refresh Token — в `httpOnly cookie` или `localStorage` с обоснованием выбора
- Автоматическое обновление Access Token через Refresh Token при истечении срока действия
- Выход из системы: очистка токенов, перенаправление на `/login`
- Подключение реальных защищённых маршрутов: `/dashboard`, `/documents/:documentId`, `/profile`
- Пользователь после логина видит только свои документы; сервер фильтрует документы по `userId` из токена
- При попытке доступа к чужому документу API возвращает 403, приложение перенаправляет на `/dashboard`

Критерии приёма

- Регистрация и вход работают корректно
- Неавторизованный пользователь при попытке открыть `/dashboard` перенаправляется на `/login`
- После входа пользователь возвращается на исходный маршрут, если он был сохранён

- Пользователь видит только свои документы
- Попытка открыть чужой документ приводит к 403 и редиректу на </dashboard>

6. Форматирование, горячие клавиши, страница профиля

Цель

Расширить функциональность ячеек, добавить полный набор горячих клавиш, страницу профиля пользователя и финально улучшить UX приложения.

Требования

- Форматирование ячеек:
 - Панель инструментов форматирования: жирный (**Ctrl+B**), курсив (**Ctrl+I**), подчёркивание (**Ctrl+U**)
 - Цвет фона ячейки и цвет текста
 - Горизонтальное выравнивание: по левому краю, по центру, по правому краю
 - Числовые форматы: обычное число, процент, валюта, дата
- Горячие клавиши:
 - **Ctrl+S** — сохранить документ
 - **Ctrl+Z** — Undo
 - **Ctrl+Y** / **Ctrl+Shift+Z** — Redo
 - **Ctrl+C** / **Ctrl+X** / **Ctrl+V** — копировать / вырезать / вставить
 - **Del** / **Backspace** — очистить ячейку
 - **Ctrl+A** — выделить все
 - **Tab**, **Enter**, **Escape** — навигация и управление редактированием
- Страница профиля:
 - Отображение имени и email текущего пользователя
 - Форма изменения имени
 - Форма смены пароля
 - Статистика: количество документов, дата регистрации

Критерии приёма

- Форматирование ячеек применяется и сохраняется после перезагрузки
- Все горячие клавиши работают корректно
- Копирование диапазона ячеек переносит и значения, и стили
- Страница профиля отображает актуальные данные; смена пароля работает
- Все состояния загрузки и ошибок обработаны
- Приложение корректно отображается на разрешениях от 1024px

Дополнительные технические требования

Качество кода

- ESLint + Prettier, тесты настроены и запускаются в CI
- Строгий режим TypeScript ("**strict**": **true** в **tsconfig.json**)
- Нет **any** — все типы явно определены или корректно выведены
- Абсолютные импорты через **paths** в **tsconfig.json**
- Нет лишних вычислений и ререндеров (используется мемоизация **useMemo**, **memo**, **useEffect** и т.д.)